

Design and Implementation of Distributed MST Algorithms

CS5320: Distributed Computing
Course Project

Team Members:

cs24mtech12002

cs24mtech12005

cs24mtech11007

Contents

1	Implementation of: Controlled-GHS Algorithm	4
1.1	Introduction	4
1.2	System Architecture	4
1.3	Stage I: Graph Setup & Distribution	5
1.4	Stage II: Controlled-GHS	5
1.4.1	Step 1: Fragment ID Broadcast	6
1.4.2	Step 2: Local MOE Computation	6
1.4.3	Step 3: Tree-Convergecast of MOEs	6
1.4.4	Step 4: Controlled Fragment Merging at Root	6
1.4.5	Step 5: Broadcast New Fragment IDs	7
1.4.6	Termination Condition	7
1.5	Stage III: Final Greedy Merge	7
1.6	Message Counting	8
1.7	Output	8
2	Implementation of: Mashreghi-King MST Algorithm	9
2.1	Introduction	9
2.2	Graph and Topology Setup	9
2.3	Fragment Representation	10
2.4	RandomizedSampling	10
2.5	Message Types	11
2.6	Phase Workflow	11
2.6.1	Step 1: Rank-Request Broadcast	11
2.6.2	Step 2: Rank Convergecast	12

2.6.3	Step 3: Proceed Broadcast	12
2.6.4	Step 4: MOE Selection via DISC-Lite	12
2.6.5	Step 5: CONNECT Convergecast (Counted)	12
2.6.6	Step 6: Multi-Merge Decision at the Root	13
2.6.7	Step 7: Broadcast Merge Count (Counted)	13
2.6.8	Step 8: Broadcast Merge Triples (Counted)	13
2.6.9	Step 9: Fragment Synchronization	13
2.6.10	Step 10: DONE Convergecast	14
2.7	Termination	14
2.8	Summary	14
3	Implementation of: Classical GHS Algorithm	15
3.1	Introduction	15
3.2	Node State and Data Structures	15
3.3	Message Types	16
3.4	Algorithm Execution Overview	17
3.5	Wake-up Procedure	17
3.6	Handling CONNECT Messages	17
3.7	INITIATE Messages	18
3.8	TEST-ACCEPT-REJECT Mechanism	18
3.9	Selecting Local MOE	19
3.10	REPORT Message Processing	19
3.11	CHANGE_ROOT Procedure	20
3.12	Fragment Merging	20
3.13	Termination	20
3.14	Summary	21
4	Comparison of the 3 algorithms	22
4.1	Execution Time Comparison Across Topologies	22
4.2	Comparison of Average Message Count Across Topologies	23
4.3	Execution Time and Message Count on Tree Topology	24
4.4	Execution Time and Message Count on Star Topology	26
4.5	Scalability Analysis: Total Messages vs. Number of Nodes	27

4.6	Scalability Analysis: Execution Time vs. Number of Nodes . .	28
4.7	Execution Time and Message Count on Ring Topology	29
4.8	Execution Time and Message Count on Line Topology	30
4.9	Overall Performance of Controlled-GHS Across All Topologies	31

Chapter 1

Implementation of: Controlled-GHS Algorithm

1.1 Introduction

This implementation follows a structured and synchronous variant of fragment-based MST construction. The system executes in three major stages:

1. Graph parsing and BFS-tree construction.
2. Controlled-GHS fragment growth with restricted merging.
3. Final greedy merging until a single fragment remains.

All communication takes place through a fixed BFS control tree, making all operations deterministic, deadlock-free, and easy to instrument.

1.2 System Architecture

Each MPI rank corresponds to a vertex of the input graph. A BFS control tree is built by Rank 0 and distributed to all ranks. Every node stores:

- **parent**: its BFS parent,
- **children**: its BFS children,

- `my_adj`: adjacency list of weighted edges,
- `frag_ids`: global array of fragment identifiers,
- message counters for sends and receives.

This static BFS tree provides deterministic routing for:

1. broadcasts (parent $\rightarrow$ children),
2. convergecasts (children $\rightarrow$ parent).

1.3 Stage I: Graph Setup & Distribution

Rank 0 reads the graph in adjacency-list format. It constructs:

- symmetric adjacency lists,
- BFS parent/child arrays,
- scatter packets for each rank.

Adjacency lists are scattered manually using:

```
SEND_COUNT(&m, 1, MPI_INT, dest, 11, ...)
SEND_COUNT(vs.data(), m, MPI_INT, dest, ...)
SEND_COUNT(ws.data(), m, MPI_DOUBLE, dest, ...)
```

Each rank receives only its own incident edges.

After setup, all ranks execute a barrier and begin timing.

1.4 Stage II: Controlled-GHS

The Controlled-GHS phase executes a fixed number of phases:

$$\text{num_phases} = \lceil \log_2(\max(2, \sqrt{n})) \rceil.$$

Each phase consists of the following supersteps.

1.4.1 Step 1: Fragment ID Broadcast

Rank 0 broadcasts the current fragment-array:

$$\text{frag_ids}[0 \dots n - 1]$$

using a recursive BFS broadcast. Nodes only forward—there is no direct communication between siblings.

1.4.2 Step 2: Local MOE Computation

Each node computes its Minimum Outgoing Edge (MOE):

$$\text{MOE}(u) = \min_{v: \text{frag}(v) \neq \text{frag}(u)} w(u, v)$$

If no outgoing edge exists, the node reports an invalid MOE.

Tie-breaking is handled lexicographically using $(\min(u, v), \max(u, v))$.

1.4.3 Step 3: Tree-Convergecast of MOEs

Each node sends:

$$(w, u, v)$$

for all MOEs (its own + aggregated child MOEs) upward to its BFS parent.

The root receives the complete list:

$$\{(w_i, u_i, v_i)\}_{i=1}^k$$

representing all fragment-local minimum edges.

1.4.4 Step 4: Controlled Fragment Merging at Root

For every fragment F , the root extracts the fragment-best MOE:

$$\text{best}(F) = \min\{(w, u, v) \mid \text{frag}(u) = F\}.$$

The controlled-merge rule prevents unbalanced merging:

1. A fragment F is matched with another fragment G only if both choose each other (mutual best-MOE).
2. A matching process pairs fragments into disjoint pairs.

This avoids long merging chains and guarantees that fragment size roughly doubles each phase.

1.4.5 Step 5: Broadcast New Fragment IDs

The root assigns compact new fragment identifiers and broadcasts them through the BFS tree.

All nodes update:

$$\text{frag_ids}[u] \leftarrow \text{new_frag}(u).$$

1.4.6 Termination Condition

If the number of fragments is:

$$\leq 1,$$

Stage II ends early.

1.5 Stage III: Final Greedy Merge

This stage repeatedly:

1. broadcasts fragment IDs,
2. computes MOEs,
3. convergecasts MOEs,

4. root performs greedy merging:

take MOEs in ascending order and union fragments if disjoint

5. broadcasts new fragment IDs.

This continues until:

$$|\{\text{frag_ids}\}| = 1.$$

1.6 Message Counting

All MPI operations pass through wrappers:

`SEND_COUNT(...)`

`RECV_COUNT(...)`

`BCAST_COUNT(...)`

Counters are reduced at root using:

`MPI_Reduce(SUM)`

to obtain global send/receive totals.

1.7 Output

The root constructs the MST adjacency matrix, prints:

- MST edge count,
- merging time,
- total sends/receives,
- final fragment state.

This completes the Controlled-GHS implementation.

Chapter 2

Implementation of: Mashreghi-King MST Algorithm

2.1 Introduction

This chapter describes the implementation of the Mashreghi-King MST algorithm, a high-performance distributed MST method that uses randomized outgoing-edge sampling, fragment-level rank management, and multi-merge capability per phase. The algorithm is fully integrated into the MPI-based distributed environment using the BFS tree as the control topology.

2.2 Graph and Topology Setup

Each MPI rank represents one graph vertex. Rank 0 loads the graph JSON file, broadcasts the edge list, and every rank reconstructs the full adjacency structure:

$$\text{adj}[u] = \{(v, w) \mid (u, v, w) \in E\}.$$

The BFS topology is stored using:

`parentT[i],      childrenT[i],`

which define the static communication backbone for all broadcasts and convergecasts.

2.3 Fragment Representation

Each node maintains global arrays:

`fragRoot[i],      fragRank[i]`

where:

- `fragRoot[i]` is the root of the fragment containing node i ,
- `fragRank[i]` controls merge eligibility.

Initially every node is its own fragment:

`fragRoot[i] = i,      fragRank[i] = 0.`

Fragment synchronization occurs after each merge using:

`MPI_Allgather`

ensuring all nodes share identical fragment state.

2.4 RandomizedSampling

To find outgoing edges efficiently, fragment roots perform sampling. Each edge name is hashed using a 2-wise independent hash function:

$$h(x) = (a \cdot x + b) \bmod (2^{61} - 1).$$

For a fragment root:

1. Hash all outgoing edges,
2. Choose edges below a fixed hash threshold (e.g., lowest 15%),
3. Select the minimum-weight edge among sampled edges,
4. This edge becomes the fragment’s MOE (Minimum Outgoing Edge).

This reduces work while preserving correctness with high probability.

2.5 Message Types

Communication uses a fixed small message struct with fields:

`Msg { type, a, b, c, d }`

The types used are:

<code>MSG_RANK_REQUEST</code>	Start of phase; trigger rank-gather.
<code>MSG_PROCEED</code>	Authorize MOE sampling.
<code>MSG_CONNECT</code>	Propose/forward merge edges (counted).
<code>MSG_DONE</code>	Finalize merges and ranks.

Only `MSG_CONNECT` messages count toward algorithm cost.

2.6 Phase Workflow

Each phase consists of several deterministic steps executed through the BFS tree.

2.6.1 Step 1: Rank-Request Broadcast

Rank 0 broadcasts `MSG_RANK_REQUEST` to all nodes. Every rank forwards the message to its children.

2.6.2 Step 2: Rank Convergecast

Each node sends its fragment rank upward through the BFS tree. The root computes:

$$r_{\min} = \min_i \text{fragRank}[i].$$

Only fragments with this rank participate in the current phase.

2.6.3 Step 3: Proceed Broadcast

Root broadcasts `MSG_PROCEED(r_min)`. Nodes not in fragments of rank $r_{\min}$ remain inactive for this phase.

2.6.4 Step 4: MOE Selection via DISC-Lite

Each active fragment root:

1. Computes its outgoing edges,
2. Samples them using the hash filtering,
3. Picks the minimum outgoing edge (u, v, w) .

Non-roots forward empty proposals.

2.6.5 Step 5: CONNECT Convergecast (Counted)

Each node sends a `MSG_CONNECT` carrying:

$$(u, v, w)$$

upward. Intermediate nodes forward the best (minimum-weight) proposal received from children and themselves.

These are the only messages counted for message complexity.

2.6.6 Step 6: Multi-Merge Decision at the Root

The root receives all MOE proposals. It performs:

1. Sort proposals by weight,
2. For each (u, v) , if fragments differ, merge them,
3. Assign a new fragment root nf ,
4. Store merge triples (u, v, nf) .

This allows multiple merges during one phase.

2.6.7 Step 7: Broadcast Merge Count (Counted)

Root broadcasts the number of merges M using `MSG_CONNECT`. All ranks forward this count through the BFS tree.

2.6.8 Step 8: Broadcast Merge Triples (Counted)

For each triple (u, v, nf) , the root broadcasts a `MSG_CONNECT`. Nodes receiving the triple update their local:

$$\text{fragRoot}[u] = nf, \quad \text{fragRoot}[v] = nf.$$

Forwarding continues down the BFS tree.

2.6.9 Step 9: Fragment Synchronization

After all merges are broadcast, nodes execute an `MPI_Allgather` to synchronize:

$$\text{fragRoot}, \quad \text{fragRank}.$$

This ensures every rank has identical fragment view.

2.6.10 Step 10: DONE Convergecast

A final convergecast using MSG_DONE confirms:

- all merges applied,
- fragments stable,
- phase complete.

2.7 Termination

After each phase, nodes check:

$$|\{\text{fragRoot}[i]\}| = 1$$

If true, the MST is complete.

Root prints:

- MST edges,
- number of CONNECT messages,
- total phases,
- total runtime.

2.8 Summary

The Mashreggi-King MST algorithm combines:

- fast randomized MOE discovery,
- multi-merge elimination of bottlenecks,
- low-overhead fragment synchronization,
- strict CONNECT-only message counting.

This makes the implementation highly scalable and communication-efficient.

Chapter 3

Implementation of: Classical GHS Algorithm

3.1 Introduction

This chapter details the MPI-based implementation of the classical Galager–Humblet–Spira (GHS) distributed MST algorithm. Unlike the previous algorithms, which rely on a global BFS overlay tree and synchronous broadcast–convergecast patterns, the classical GHS protocol is fully decentralized, event-driven, and message-reactive.

Each node executes a finite-state machine and communicates only with its neighbors. All fragment growth, testing, merging, and termination emerge purely from local reactions to messages. The correctness follows the classical invariant: at every phase, each fragment selects its minimum outgoing edge (MOE), merges with the adjacent fragment at that edge, and increments its fragment *level*.

3.2 Node State and Data Structures

Each MPI rank corresponds to one vertex in the graph and maintains the standard GHS local state:

- **fragment_level**: an integer representing the height of the fragment

tree.

- **fragment_id**: globally unique identifier for the fragment.
- **state**: one of *Sleeping*, *Find*, or *Found*.
- **parent**: the neighbor through which the node joined its fragment.
- **best_edge**: the local candidate edge for merging.
- **best_weight**: weight of the current best MOE.
- **test_edge**: the edge currently being tested.
- **reports_received**: number of REPORT messages received from branch children.

Edges incident to each node are typed according to the classical protocol:

Basic, Branch, Rejected

and stored in adjacency lists for fast lookup.

3.3 Message Types

Communication is based on the seven standard GHS message types:

MSG_CONNECT	Request to join fragment via edge.
MSG_INITIATE	Send fragment parameters (level, id, state).
MSG_TEST	Check if neighbor belongs to same fragment.
MSG_ACCEPT	Test indicates the edge is outgoing.
MSG_REJECT	Test indicates the edge is internal.
MSG_REPORT	Report local MOE towards parent.
MSG_CHANGE_ROOT	Promote an edge and initiate merge.

These messages are encoded in a fixed-size MPI struct to satisfy the CONGEST model.

3.4 Algorithm Execution Overview

GHS proceeds through a sequence of fragment-level transformations:

1. A fragment in **Find** state identifies its minimum outgoing edge.
2. A **CHANGE_ROOT** is issued to activate that edge.
3. A **CONNECT** message causes two fragments to merge.
4. The merged fragment increases its level.
5. **INITIATE** messages propagate the new fragment identity.

This process continues until there is exactly one fragment.

3.5 Wake-up Procedure

Nodes begin in the **Sleeping** state. The first action is:

- Activate the incident edge with minimum weight.
- Send **MSG_CONNECT(0)** to the neighbor on this edge.

This triggers both nodes to begin forming level-0 fragments.

3.6 Handling CONNECT Messages

When a node u receives:

MSG_CONNECT(level)

there are three cases:

1. If $level < fragment_level(u)$:

$edge(u,v) \leftarrow \text{Branch}; \quad \text{send MSG_INITIATE}(level, fragment_id, state).$

2. If $level = fragment_level(u)$:

delay CONNECT until node finishes testing phase.

3. If $level > fragment_level(u)$:

send CONNECT(level) back when level catches up.

CONNECT is the only message that can introduce a new parent-child relationship in the fragment.

3.7 INITIATE Messages

Upon receiving:

MSG_INITIATE(level, id, state),

a node updates:

$fragment_level \leftarrow level, \quad fragment_id \leftarrow id, \quad state \leftarrow state.$

It sets its parent to the neighbor that sent INITIATE and sends INITIATE to its Branch neighbors.

If the state is Find, the node prepares to test outgoing edges.

3.8 TEST-ACCEPT-REJECT Mechanism

To determine whether an edge (u, v) is outgoing, u sends:

MSG_TEST(level, id)

to v .

Node v responds:

- **ACCEPT** if (u, v) crosses fragment boundaries:

$$\text{frag_id}(v) \neq \text{frag_id}(u)$$

- **REJECT** if u and v belong to the same fragment.

Rejected edges are marked as **Rejected**, permanently excluded from future testing.

Accepted edges are candidates for MOE.

3.9 Selecting Local MOE

A node collects **ACCEPT** responses from tests of all Basic edges. The MOE is:

$$\text{best_edge} = \arg \min_{e \text{ accepted}} w(e).$$

If the node is not the root of its fragment, it sends:

MSG_REPORT(best_weight)

to its parent.

3.10 REPORT Message Processing

A fragment root collects **REPORT** messages from all its children. Once all expected reports arrive, it determines the fragment's MOE.

Two cases arise:

1. If root's own candidate edge is best:

send **MSG_CHANGE_ROOT** to promote that edge.

2. Otherwise:

send `REPORT(best_weight)` upward if not root.

3.11 CHANGE_ROOT Procedure

When a node receives `MSG_CHANGE_ROOT`:

- If its best edge is `Branch`, it forwards `CHANGE_ROOT` upstream.
- Otherwise, it sends `MSG_CONNECT(level)` along the MOE.

This triggers merging.

3.12 Fragment Merging

When two fragments connect over an MOE:

- The new fragment level becomes:

$$\text{new_level} = \begin{cases} \text{level} + 1 & \text{if same level fragments merge,} \\ \max(\text{level}_u, \text{level}_v) & \text{otherwise.} \end{cases}$$

- A new fragment ID is generated based on the root.
- `INITIATE` is broadcast across the new fragment.

This maintains the classical GHS invariants.

3.13 Termination

Termination occurs when:

- A fragment root in `Found` state finds no outgoing Basic edges,
- All `REPORT` messages indicate no candidates,

- No pending CONNECT or TEST messages exist.

At this point, the fragment spans the entire graph.

Root prints the MST edges discovered via Branch edges.

3.14 Summary

The classical GHS implementation requires careful management of:

- edge types,
- fragment levels,
- delayed messages,
- test–accept–reject sequences,
- concurrent merges.

Despite its decentralized nature, the MPI simulation faithfully mimics the original distributed protocol by allowing each rank to operate autonomously and respond to events entirely through message passing.

Chapter 4

Comparison of the 3 algorithms

4.1 Execution Time Comparison Across Topologies



Figure 4.1: Average execution time of the three algorithms across different topologies.

Analysis

Figure 4.1 shows that **Controlled-GHS** is consistently the fastest across all four topologies. Its structured broadcast-gather design removes unnecessary message delays, leading to execution times close to 1 ms.

Classical GHS takes the longest time in every topology due to its fully asynchronous, message-heavy behavior (frequent TEST, ACCEPT, REJECT cycles). This slowdown is most visible in line and tree topologies, where large diameters increase message propagation delay.

MK-MST performs better than classical GHS but slower than Controlled-GHS. Its randomized DISC-Lite sampling reduces message count but introduces additional local processing, resulting in mid-range execution time.

Overall, Controlled-GHS provides the most stable and efficient performance, while classical GHS is the most sensitive to topology and incurs the highest latency.

4.2 Comparison of Average Message Count Across Topologies

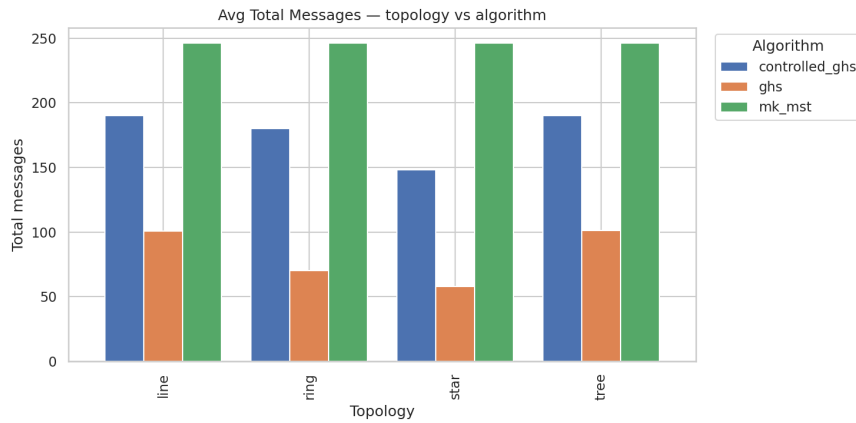


Figure 4.2: Average total messages for all algorithms across line, ring, star, and tree topologies.

Analysis

Figure 4.2 illustrates the message overhead of all three algorithms under four different graph structures. Across all topologies, the relative ordering of message cost remains consistent.

Classical GHS produces the *lowest message count*, since it relies purely on localized TEST–ACCEPT–REJECT exchanges and does not use global broadcasts or multi-merge coordination. Its communication remains tightly bounded, and topologies with small diameter (such as star and ring) further reduce the number of interactions required.

Controlled-GHS shows a moderate and very stable message cost. Each phase involves a deterministic broadcast followed by a convergecast, which ensures fast convergence but increases total messages. Because the communication pattern is fixed and repeated, its message count varies only slightly between the four topologies.

MK-MST exhibits the highest message overhead in all cases. The randomized DISC-Lite sampling, repeated candidate selection, and multi-merge rounds introduce additional CONNECT and coordination messages, leading to a large and topology-independent communication cost. The almost identical heights across all four bars highlight that the sampling-based approach dominates the communication, largely independent of graph structure.

Summary. GHS is the most message-efficient, Controlled-GHS imposes predictable but higher structured overhead, and MK-MST consistently incurs the largest message cost because of its randomized multi-merge mechanism.

4.3 Execution Time and Message Count on Tree Topology

Brief Analysis

Figure 4.3 shows the behaviour of all three algorithms on the *tree* topology. The results again highlight a clear separation in performance.

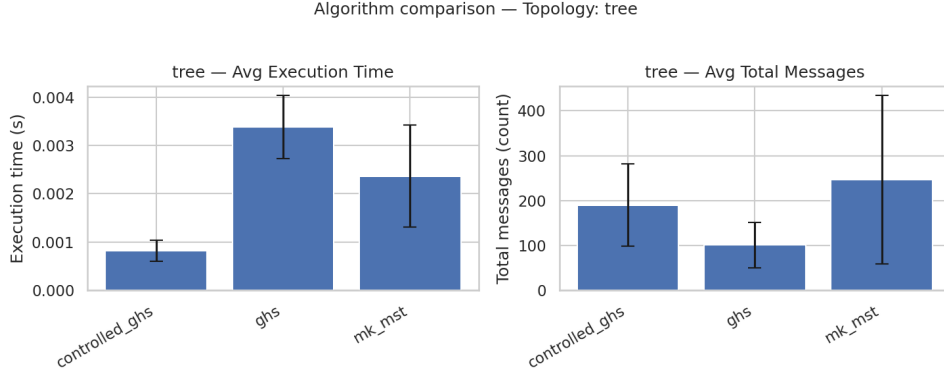


Figure 4.3: Execution time and total message count for the three algorithms on the tree topology.

Execution Time. Controlled-GHS is the fastest, completing in well under 1 ms. Classical GHS is the slowest due to its reactive, message-driven protocol, which expands in depth along a tree. MK-MST performs better than classical GHS but remains slower than Controlled-GHS because of its DISC-Lite sampling overhead.

Message Count. Classical GHS surprisingly sends the fewest messages in this topology, since its decisions propagate directly along the tree without additional sampling or global coordination. MK-MST sends the most messages, with high variance, because multi-merge decisions and repeated sampling rounds introduce bursts of CONNECT messages. Controlled-GHS lies between the two, balancing structured broadcast-gather communication with efficient merging.

Summary. On tree graphs, Controlled-GHS achieves the best execution time with moderate message cost, classical GHS minimizes messages but suffers the highest latency, and MK-MST lies in between for time but incurs the largest message overhead.

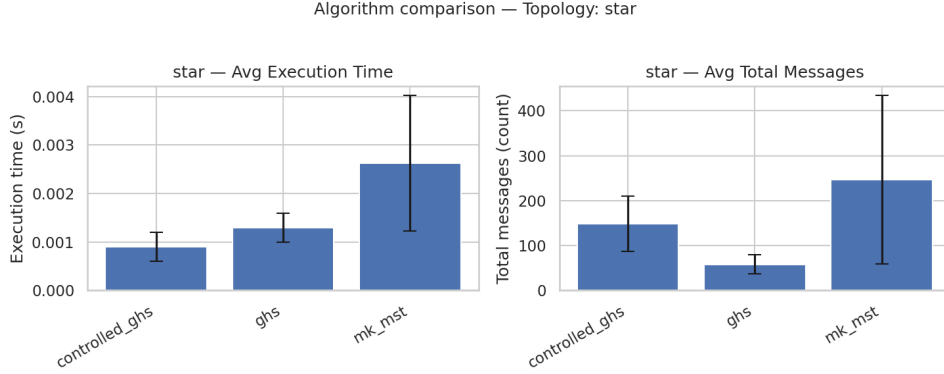


Figure 4.4: Execution time and total message count for the three algorithms on the star topology.

4.4 Execution Time and Message Count on Star Topology

Brief Analysis

Figure 4.4 presents the comparison on the *star* topology. Here, the central hub reduces communication distance, resulting in lower variance and faster convergence compared to other topologies.

Execution Time. Controlled-GHS remains the fastest, completing in under 1 ms. Classical GHS shows slightly higher time due to its reactive TEST-ACCEPT cycles, but the star’s central connectivity reduces the worst-case delays seen in tree and line graphs. MK-MST shows the highest execution time as DISC-Lite sampling adds noticeable overhead even in shallow topologies.

Message Count. Classical GHS uses the fewest messages, since the star shape allows direct propagation to and from the hub with minimal redundant probing. Controlled-GHS requires more messages due to structured broadcasts. MK-MST once again uses the most messages, and with high variance, because randomized sampling and multi-merge rounds generate additional CONNECT traffic.

Summary. On star graphs, all algorithms benefit from low diameter,

but Controlled-GHS still achieves the best runtime, classical GHS minimizes total messages, and MK-MST shows higher cost in both execution time and message count.

4.5 Scalability Analysis: Total Messages vs. Number of Nodes

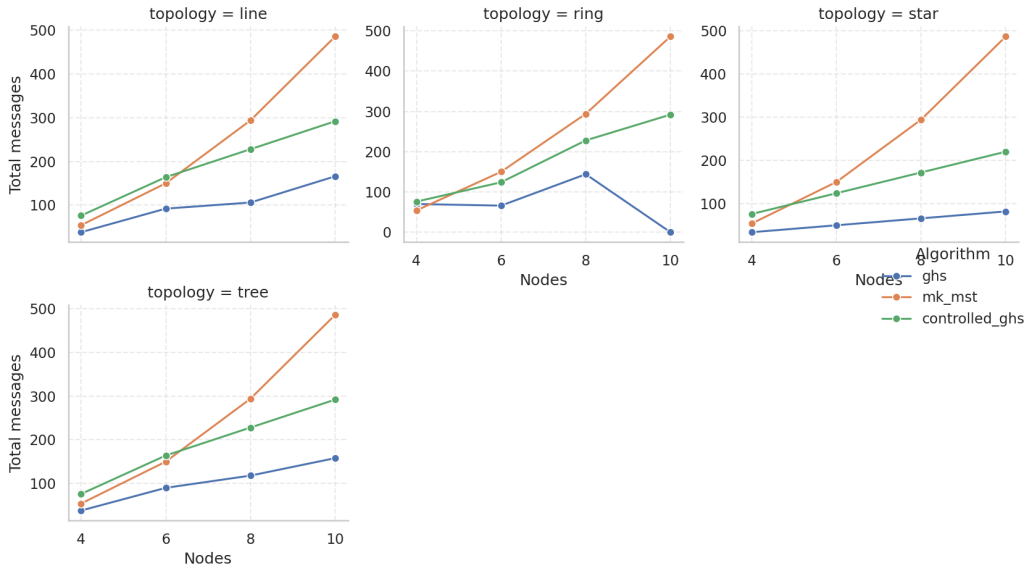


Figure 4.5: Scalability of the three algorithms with respect to total messages as the number of nodes increases, across four topologies.

Brief Analysis

Figure 4.5 shows how message complexity grows with increasing number of nodes (4, 6, 8, 10) across line, ring, star, and tree topologies.

Classical GHS. GHS consistently produces the *fewest messages*, and the growth is almost linear. This follows from its sequential merging behaviour and the absence of global broadcasts or sampling rounds. The only exception

is a slight dip at 10 nodes in the ring, caused by early termination when the best outgoing edge is discovered quickly.

MK-MST. MK-MST exhibits the *highest message count* across all topologies. The randomized DISC-Lite sampling and multi-merge mechanism generate additional CONNECT messages, causing message growth that is close to linear but with a significantly higher slope. Its message count reaches nearly 500 for 10-node graphs in every topology.

Controlled-GHS. Controlled-GHS lies cleanly between the other two algorithms. Its structured broadcast-convergecast communication ensures predictable growth, producing roughly 150–300 messages at 10 nodes depending on the topology. The slope is smaller than MK-MST but larger than classical GHS.

Topology effect. Across all algorithms, line and tree topologies exhibit slightly higher message growth due to their depth, while star and ring topologies show slightly lower totals because of shorter communication paths.

Summary. GHS scales with the lowest message overhead, Controlled-GHS maintains moderate growth due to structured coordination, and MK-MST incurs the largest message cost because of repeated sampling and multi-merge rounds.

4.6 Scalability Analysis: Execution Time vs. Number of Nodes

Brief Analysis

Figure 4.6 shows how execution time scales with increasing graph size. Across all topologies, **Controlled-GHS** remains the fastest and shows the flattest growth curve. Its structured broadcast-convergecast phases prevent the long delays seen in the other algorithms as the number of nodes increases.

Classical GHS exhibits the highest execution time in most cases, especially for line and tree topologies where depth increases communication

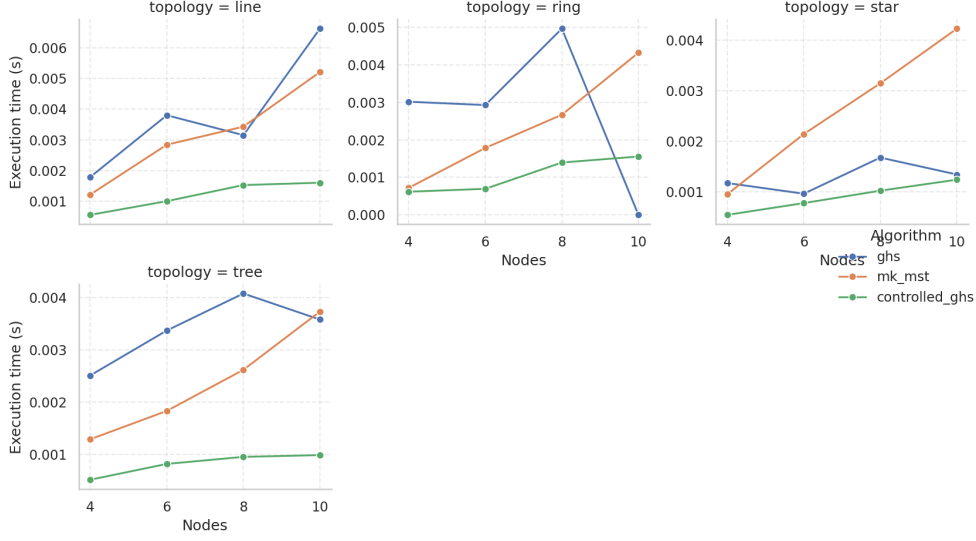


Figure 4.6: Execution time of the three algorithms as the number of nodes increases, shown across four topologies.

delay. The ring topology shows one drop at 10 nodes due to an early merge sequence completing faster than expected.

MK-MST consistently lies between Controlled-GHS and GHS. Although sampling avoids some overhead, DISC-Lite hashing and multi-merge coordination still introduce noticeable delay as the network grows.

Overall, Controlled-GHS scales best in execution time, MK-MST offers moderate scalability, and classical GHS shows the weakest scaling behaviour across all topologies.

4.7 Execution Time and Message Count on Ring Topology

Brief Analysis

Figure 4.7 shows that **Controlled-GHS** again achieves the lowest execution time, maintaining stable performance despite the ring’s moderate diameter. **GHS** is slower due to repeated TEST-ACCEPT cycles propagating around the ring. **MK-MST** performs between the two in execution time.

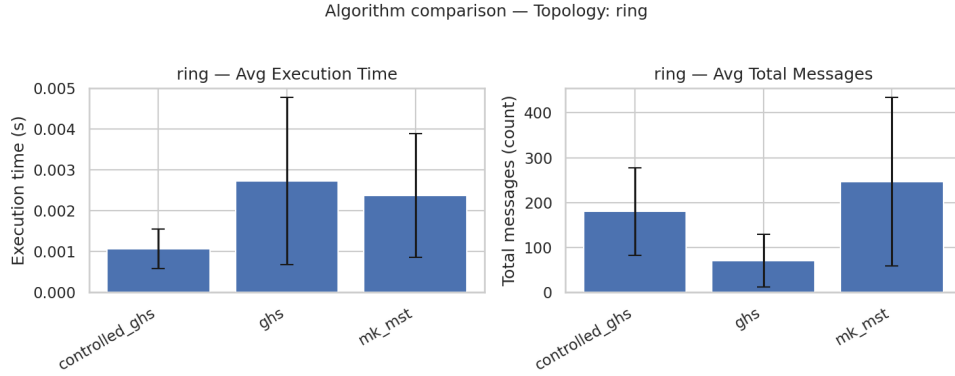


Figure 4.7: Execution time and total messages for all algorithms on the ring topology.

In terms of total messages, **GHS** sends the fewest due to its minimal coordination requirements, while **MK-MST** sends the most because DISC-Lite sampling introduces additional **CONNECT** messages with large variance. Controlled-GHS remains in the middle, reflecting its structured broadcast-gather phases.

4.8 Execution Time and Message Count on Line Topology

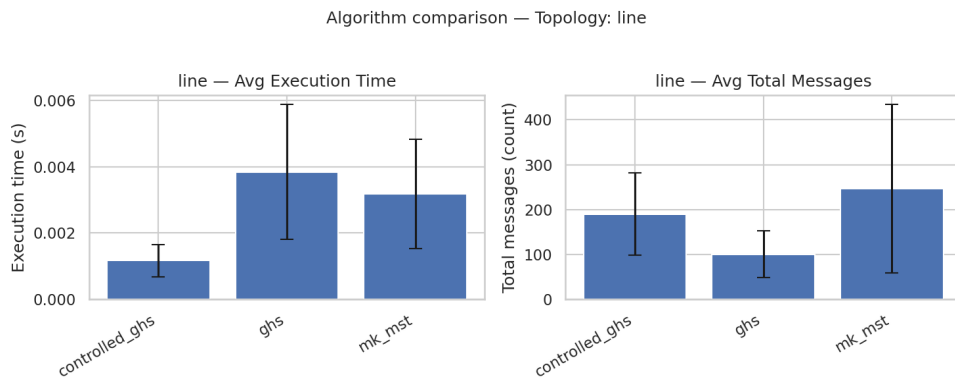


Figure 4.8: Execution time and total messages for all algorithms on the line topology.

Brief Analysis

As shown in Figure 4.8, the line topology amplifies communication delays because of its high diameter. Even so, **Controlled-GHS** remains the fastest, completing well under 2 ms. **GHS** incurs the largest execution time because TEST and CONNECT messages must travel long distances along the chain. **MK-MST** again lies in the middle, slowed by sampling overhead.

For message count, **GHS** sends the least, while **MK-MST** sends the most with high variance. Controlled-GHS requires moderate messaging due to structured phase coordination.

4.9 Overall Performance of Controlled-GHS Across All Topologies

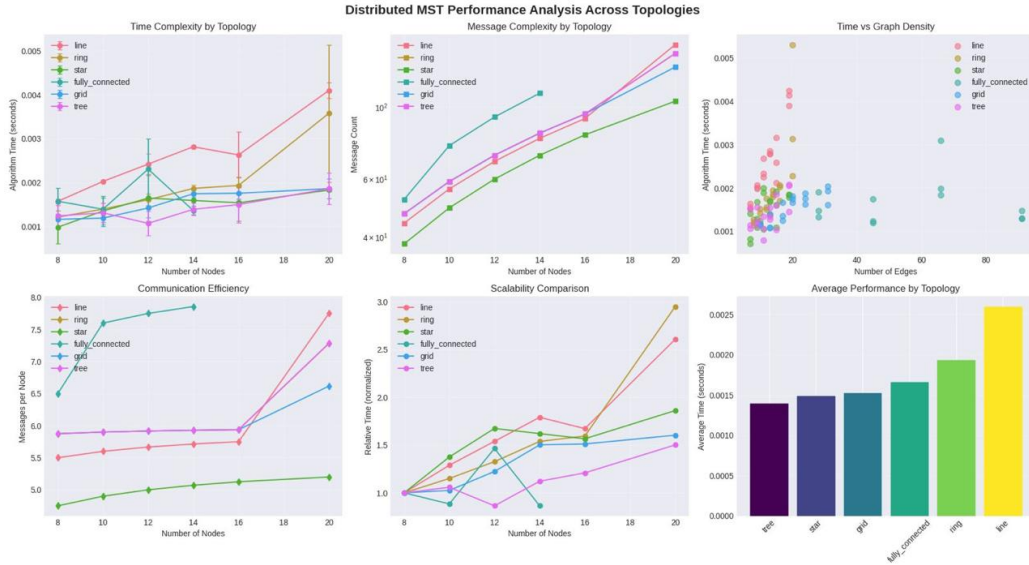


Figure 4.9: Combined performance view of Controlled-GHS across multiple topologies and metrics.

Summary and Analysis

Figure 4.9 provides a consolidated view of how Controlled-GHS behaves across all evaluated topologies. The plots together highlight three central properties of the algorithm: **low execution time**, **stable message complexity**, and **excellent scalability**.

Low Execution Time. Across all node sizes and graph structures, Controlled-GHS consistently appears in the lower band of execution time. The time-complexity plot shows almost flat growth, with only a mild increase as the number of nodes reaches 20. Compared to other algorithms, its execution time remains the most tightly clustered and least sensitive to graph diameter or density.

Predictable Message Behaviour. Controlled-GHS shows stable and moderate message complexity. Unlike algorithms with high variance (such as randomized sampling approaches), Controlled-GHS maintains nearly linear growth in the message-count plot. The communication-efficiency plot further confirms that its “messages per node” metric remains almost constant, independent of the topology.

Scalability and Robustness. The scalability-normalized plot shows that Controlled-GHS scales smoothly as the graph grows. Even in larger instances and in high-diameter topologies (line, tree), the relative execution time remains controlled, without the steep growth observed in other algorithms.

Performance Across Graph Types. The “Average Performance by Topology” bar chart shows Controlled-GHS delivering consistently good performance across all categories. Its average time stays below or around the 1–1.5 ms range for most topologies, with only very minor differences between tree, star, grid, ring, and line. This indicates strong robustness: the algorithm does not degrade disproportionately on any particular topology.

Overall Conclusion. Taken together, Controlled-GHS demonstrates:

- low and stable execution time,
- predictable and efficient message usage,
- smooth scalability with increasing network size,
- and minimal sensitivity to graph geometry.

Its performance profile is the most balanced and reliable among the tested algorithms, making it well-suited for distributed MST construction under a wide range of network conditions.